Codes de Reed-Solomon

Implémentation C++ d'un Codeur et Décodeur RS

Projet thématique

Khalil Bitar - Nasr-allah Hitar

ENSEIRB

2025/2026

Sommaire

1. Présentation du projet
2. Description du système actuel
3. Objectifs

Présentation du projet

Contexte

Les codes de **Reed–Solomon** (RS) sont des codes correcteurs d'erreurs utilisés dans de nombreuses applications critiques : CD/DVD, codes QR, transmissions spatiales, 4G/5G...

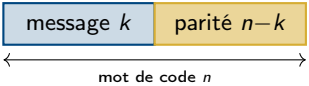
Objectif général du projet

Implémenter en C++ un **codeur et décodeur Reed–Solomon complet** :

- Priorisation de performance
- Codeur optimisé
- Décodeur systématique

Notations principales

Code	$\mathrm{RS}(n, k)$
Corps Galois	$\mathrm{GF}(2^m)$
Erreurs	$t = \frac{n-k}{2}$
Distance min.	$d = n - k + 1$



Corps de Galois et Encodeur

GaloisField — $\mathrm{GF}(2^m)$

- Construit à partir d'un polynôme primitif
- Tables `alpha_to[i]` $= \alpha^i$ et `index_of[x]` $= \log_\alpha x$

Encodeur systématique

Polynôme générateur : $g(x) = \prod_{i=1}^{n-k}(x - \alpha^i)$

Mot de code :
$c(x) = x^{n-k}m(x) - \left(x^{n-k}m(x) \bmod g(x)\right)$
$\Rightarrow \left(x^{n-k}m(x) \bmod g(x)\right)$ étant le reste de la division euclidienne

Test RS(7,3) sur GF(8)

Message : $[2, 4, 3]$ $(= \alpha^1, \alpha^2, \alpha^3)$

parité				message		
r_0	r_1	r_2	r_3	2	4	3

Syndromes

Syndromes $S_i = r(\alpha^i)$ calculés par Hörner
$\Lambda(x)$ obtenu par **Berlekamp–Massey**
$\Rightarrow$ localisateur d'erreurs prêt pour Chien

Décodeur — Ce qui est implémenté

Étape 1 : Syndromes $S_i = r(\alpha^i)$

Évaluation de $r(x)$ aux racines de $g(x)$.
Si $\forall i,\ S_i = 0$: pas d'erreur, sortie immédiate.

Étape 2 : Berlekamp–Massey

Construit itérativement $\Lambda(x)$, le polynôme localisateur d'erreurs, en minimisant la longueur du LFSR générant la suite des syndromes.

$$d = S_r + \sum_{i=1}^{L} \Lambda_i\, S_{r-i}$$

Mise à jour : si $d \neq 0$, correction de $\Lambda(x)$.

Sortie de Berlekamp–Massey

$\Lambda(x)$ de degré ν = nombre d'erreurs

Exemple ($\nu = 1$ erreur) :
$\Lambda(x) = 1 + \Lambda_1 x$
où $\Lambda_1^{-1} = X_1 = \alpha^{p_1}$ localise l'erreur.

Prochaine étape immédiate

1. **Recherche de Chien** :
   évaluer $\Lambda(\alpha^i)$ pour $i = 0 \ldots n-1$
2. **Forney** :
   calculer $e_k = \dfrac{X_k \cdot \Omega(X_k^{-1})}{\Lambda'(X_k^{-1})}$

Décodeur

Étape 1 : Syndromes $S_i = r(\alpha^i)$

Évaluation de $r(x)$ aux racines de $g(x)$.
Si $\forall i,\ S_i = 0$: pas d'erreur, sortie immédiate.

Étape 2 : Berlekamp–Massey

Construit itérativement $\Lambda(x)$, le polynôme localisateur d'erreurs, en minimisant la longueur du LFSR générant la suite des syndromes.

$$d = S_r + \sum_{i=1}^{L} \Lambda_i\, S_{r-i}$$

Étape 3 : Recherche de Chien

Trouve les positions d'erreurs en évaluant les racines du polynôme localisateur :
$\Lambda(\alpha^{-i}) = 0 \Rightarrow$ erreur à la position i.

Étape 4 : Formule de Forney

Calcule l'amplitude exacte de l'erreur Y_k pour corriger le symbole :

$$Y_k = \frac{X_k \cdot \Omega(X_k^{-1})}{\Lambda'(X_k^{-1})}$$

Note : Le signe négatif est ignoré.

Objectifs : Optimisation Logicielle

1. Analyse de performances du code C++

- Mesure de la complexité pratique des blocs critiques (Syndromes, Berlekamp-Massey $O(t^2)$, Chien).
- Identification d'améliorations possibles au niveau de performance.

2. Pistes d'optimisation algorithmique

- Amélioration de la fonction de multiplication polynomiale.
- Étude d'intégration d'une Transformée de Fourier Rapide (FFT) sur des corps finis pour accélérer l'évaluation polynomiale (butterfly).

Objectifs : Transition Matérielle

Intégration RISC-V et FPGA

- **Portage :** Cross-compilation du code C++ optimisé pour l'architecture cible RISC-V.
- **Co-design :** Début de la conception de modules matériels (en VHDL ou HLS) pour décharger le processeur.
- **Validation :** Préparation d'un démonstrateur matériel pour analyser les performances in situ.